PRODUCT & TECHNICAL OVERVIEW

OpenNoteBox

A self-hosted, privacy-first document intelligence workspace -
read, chat, listen, and memorize, on your own hardware, with your own model.

One sentence: OpenNoteBox is a self-hosted NotebookLM-style reader for PDF, EPUB, DOCX, PPTX and Markdown that adds AI summaries, citation-grounded chat (hybrid BM25 + embeddings + knowledge-graph retrieval), word-synced TTS narration, two-host AI podcasts, and FSRS spaced-repetition flashcards - all running against any OpenAI-compatible LLM endpoint you control.

Self-hosted

Your documents never leave hardware you control. No vendor lock-in, no per-seat license.

Model-agnostic

Ollama, vLLM, llama.cpp or any hosted OpenAI-compatible API, switchable from the UI.

Full lifecycle

One tool for reading, asking, listening and reviewing - not four subscriptions.

Local inference

GPU or CPU TTS (Piper / Kokoro-82M); works fully offline with a local LLM.

Repository: `open-notebox` | Status: pre-1.0, actively developed | License: open source (MIT) | Report date: 2026-09-06

1. Problem Statement

Professionals, researchers and students live inside long documents - papers, textbooks, reports, manuals, books - and today they stitch that workflow together from 3-5 disconnected paid products: one for reading, one for "chat with PDF", one for highlights, one for flashcards, one for read-aloud. That stitching has four structural costs:

PROBLEM	WHAT IT LOOKS LIKE IN PRACTICE
Privacy trade-off	Hosted "chat with your PDF" tools require uploading confidential reports, unpublished research, patient-adjacent material or licensed books to someone else's servers - usually under ToS that permits model training or indefinite retention.
Fragmentation	Understanding a document means re-finding it in four tools: the reader (highlights), the chat tool (questions), the flashcard app (memorization), the TTS app (commute listening). Nothing is shared between them; citations break at every hop.
Trust / hallucination	Generic LLM chat gives confident answers untethered from the source. What makes a document tool useful is a citation you can click that lands on the exact page.
Cost & continuity risk	Per-seat SaaS pricing multiplies across a team or household; features change or disappear, APIs get repriced, and your accumulated notes/highlights/schedules become hostage to the vendor.

OpenNoteBox's answer: one deployed container stack that owns the entire document lifecycle - ingest, parse, index, ask, narrate, review - on your infrastructure, talking to a model you chose, with citations that land on the source page and a spaced-repetition schedule (the algorithm behind modern Anki) built into the same app.

2. What the Product Does

Read

- Multi-format reader: PDF (page-image + selectable text layer, column-aware reading order, per-document layout modes for research papers), EPUB, DOCX, PPTX, TXT, Markdown - all normalized to one page/section model.
- Highlights in multiple colors with notes, tags, page-range anchors that survive re-parsing; bookmarks and per-document reading position.
- Automatic section/chapter detection via PDF outline, heading heuristics, or research-paper structure.

Ask

- Chat grounded in one or many documents with numbered inline citations that jump to the source page.
- Three retrieval modes: standard (BM25 + embeddings + reranking, with HyDE-style query rephrasing), hybrid (vectors + knowledge graph), and graph-only (entity/relationship traversal with hub-node fallback).
- Per-page and per-section AI summaries, generated on demand, cached, with status tracking.

Listen

- TTS narration of any page or summary with **word-level synced highlighting** in the reader; page furniture (numbers, running headers, tables) filtered from speech.
- Two-host podcast generation: any document or section becomes a male/female dialogue in distinct voices, synthesized once and cached as audio.
- Self-hosted engines (Piper CLI, Kokoro-82M ONNX), CPU or CUDA; GPU is released back when idle.

Remember

- AI-generated flashcards from document content, or one-click cards from any highlight.
- Review queue driven by **FSRS** - the open-source spaced-repetition scheduler that is Anki's default since 2023 - with cloze and basic/reversed card types, stats and daily quotas.

Administer

- Shared LLM-backend registry (admin-managed), per-user preferences, user management, metrics dashboard, deep health checks, opt-in export mirror to disk.
- Live processing progress pushed over WebSocket instead of polling.

3. Technical Stack

LAYER	TECHNOLOGY	NOTES
Frontend	Next.js 14 (App Router) · React 18 · TypeScript · Tailwind CSS · Radix UI · TanStack Query · Zustand	pdf.js 4 viewer with custom text layer for TTS sync and highlight anchoring; react-markdown (raw HTML disabled) for LLM output.
API	Python 3.11+ · FastAPI · Pydantic v2 · WebSockets	JWT (HS256) auth with access/refresh separation; bcrypt; per-router ownership checks.

LAYER	TECHNOLOGY	NOTES
Async work	Celery (Redis broker) · separate queues: parsing/summaries, TTS/podcasts, beat scheduler	Latency-sensitive narration never queues behind bulk re-parses; per-task time limits + zip-bomb guards.
Data	PostgreSQL 16 (SQLAlchemy 2, Alembic migrations) · Redis	Full-text search via native `tsvector`/GIN (BM25-like) plus embedding storage; no separate vector DB to operate.
Retrieval / LLM	LangChain / LangGraph · tiktoken · networkx (knowledge graph) · httpx	Model-agnostic: any OpenAI-compatible endpoint (Ollama, vLLM, llama.cpp, hosted APIs). Hybrid BM25+vector retrieval, HyDE rephrasing, reranking, KG mode.
Document parsing	pdfplumber (layout, columns, OCR hooks) · python-docx · python-pptx · ebooklib · BeautifulSoup	Normalized page/section model with bounding boxes; magic-byte validation on URL imports.
TTS	Custom FastAPI service · Piper + Kokoro-82M (ONNX), optional CUDA	Word-level timestamps drive reader highlighting; LRU audio cache with eviction.
Scheduling / learning	py-fsrs (FSRS-6 spaced repetition)	Same algorithm family as Anki; per-user daily limits.
Infrastructure	Docker Compose · Traefik 3 (single HTTPS origin, path-routed) · optional Cloudflare Tunnel	All infra ports loopback-bound; GPU compose overlay; startup refusal on placeholder JWT secret.
Quality	Pytest (285 tests), Playwright e2e, benchmark scripts, resource/DB-pool monitoring	Regression tests for audit fixes; live system metrics endpoints.

4. Competitive Landscape

4.1 Commercial hosted products

PRODUCT	POSITIONING	KEY STRENGTHS	WHERE OPENNOTEBOX DIFFERS
Google NotebookLM (Gemini Notebook)	The category-definer: source-grounded research assistant + viral two-host "Audio Overviews".	Google's best models; polish; multi-source notebooks (PDF, docs, video, web); Audio Overviews; free tier; web/desktop/mobile.	Fully hosted - your sources sit on Google. OpenNoteBox matches the core loop (citations + audio dialogue + flashcards) with your own GPU and model; adds word-synced reading narration, in-reader highlights/notes, and FSRS review, which NotebookLM does not do.
ChatPDF / ChatGPT-file uploads	Simplest "chat with this PDF" utility; free tier is 2 docs/day.	Zero setup; instant; good-enough answers.	Single-document chat, no reading experience, no highlight/podcast/flashcard lifecycle, and every page is uploaded to the vendor. OpenNoteBox targets the whole study/retention loop, offline-capable.
Elicit / Scholarcy	Research-paper-specific assistants (screening, extraction, summaries).	Deep paper workflows; academic search integrations; team features.	Paper-only and hosted. OpenNoteBox is general (books, manuals, reports) and self-hosted, though it currently lacks literature-catalog integrations.
Readwise Reader	Read-later app with highlight sync + AI ask.	Best-in-class highlight pipeline across devices/apps; newsletter/web clipping.	No local inference, weaker long-PDF layout handling, subscription per user. OpenNoteBox trades the clip-everything breadth for depth on uploaded files + audio + review.

4.2 Open-source / self-hosted alternatives

PROJECT	STARS	FOCUS	VS. OPENNOTEBOX
AnythingLLM (Mintplex)	~40k	Desktop/server RAG workspace, workspaces, agents, embeddable chat.	Broader (agents, connectors) but no real reader: no page-level PDF layout, no synced TTS, no FSRS. OpenNoteBox is reader-first.
Open WebUI	~100k	Chat UI for Ollama et al. with RAG doc uploads.	Chat-centric; documents are context, not a product surface. No citations-to-page, no narration/study features.
RAGFlow	~60k	Enterprise RAG engine, deep document parsing, agent workflows.	Parsing is strong, but it is a dev platform, not an end-user reading/study app; heavier stack (Elasticsearch etc.).
Khoj	~25k	Personal AI over notes/docs, agentic search, WhatsApp/web UI.	Notes-first (Org-mode/Obsidian), not PDF-reader-first; no TTS sync or podcasts.
Open Notebook / NotebookLM-Lite / Memorwise and similar clones	<1k each	Self-hosted NotebookLM lookalikes: RAG chat + citations, some with podcast gen.	Closest in spirit; mostly thin Next+FastAPI demos. OpenNoteBox's differentiators vs. this field: production page-image PDF viewer with layout modes, word-synced narration, KG retrieval mode, FSRS, security hardening, 285-test suite.
Anki (adjacent)	-	The reference FSRS flashcard app.	Unmatched review tooling but zero document understanding. OpenNoteBox auto-generates and anchors cards from the actual reading - the bridge Anki doesn't have.

4.3 Capability matrix

CAPABILITY	OPENNOTEBOX	NOTEBOOKLM	CHATPDF	ANYTHINGLLM	RAGFLOW
Self-hosted / air-gapped	Yes	No	No	Yes	Yes
Your own model choice	Any OpenAI-compat	Google only	Vendor	Yes	Yes
Real PDF reader (page images + layout)	Yes	Partial	Text only	No	Partial
Citations land on source page	Yes	Yes	Text cite only	Chunk-level	Chunk-level
Knowledge-graph retrieval mode	Yes	No	No	Agents/DAGs	Workflows
TTS narration synced to reader	Word-level	No	No	No	No

Two-host AI podcast (self-hosted voices)	Yes	Yes (hosted)	No	No	No
FSRS spaced-repetition built in	Yes (py-fsrs)	Flashcards, basic	No	No	No
Highlights/notes anchored in document	Yes	Notes only	No	No	No
Mobile app / responsive	Desktop-first	Yes	Yes	Desktop/PWA	Web
Multilingual TTS	English voices bundled	Dozens	Hosted voices	BYO	BYO
Cost model	Free + your HW	Free tier / sub	Freemium	Free / paid desktop	Free

Capability assessments for third-party products reflect public information as of Sep 2026 and simplify; verify before quoting. OpenNoteBox column reflects the codebase itself.

4.4 Positioning summary

OpenNoteBox occupies a niche none of the big players cover: **the full read-ask-listen-review loop, self-hosted, with a real document viewer**. The hosted giants (NotebookLM) win on model quality, polish and mobile; the open-source RAG platforms (AnythingLLM, RAGFlow, Open WebUI) win on generality and integrations but treat documents as context blobs. OpenNoteBox's moat is the integration depth between reading, citations, synced narration and FSRS - plus privacy by architecture rather than policy.

5. Pros & Cons

Pros

Privacy by architecture

- No document ever leaves your infrastructure; works fully air-gapped with local Ollama/vLLM.
- No training-data clause, no vendor ToS risk, no per-seat cost.

Product differentiation

- One tool replaces reader + chat-PDF + TTS + flashcards; citations are the connective tissue (chat → page, highlight → flashcard).
- Word-level synced narration and two-host podcasts on self-hosted TTS - rare outside Google's hosted stack.
- FSRS scheduling (Anki-grade) built in; cards regenerate from real content, not manual authoring.
- Three retrieval modes (BM25+vector, hybrid, knowledge graph) with HyDE-style query rewriting.

Engineering quality

- Sensible architecture: queue separation (narration never waits behind re-parses), Alembic migrations, single-origin TLS via Traefik, no cross-origin requests.
- Security posture well above typical self-hosted level (independent audit): placeholder-JWT boot refusal, SSRF IP-pinning, zip-bomb guards, rate-limited auth, consistent per-object ownership checks (no IDOR found), no raw-SQL injection surface.
- 285-test backend suite incl. Playwright e2e; operational monitoring (DB pool, GPU/system stats, LLM health).
- Genuinely model-agnostic; backend registry switchable at runtime without redeploy.

Cons & Risks

Maturity (pre-1.0)

- Single-deployment design: no team features, sharing, SSO/invites, or per-user storage quotas; designed for one person to admin one instance.
- Desktop-only layout; no mobile app; English-only bundled TTS voices.
- Product-surface test coverage acknowledged thin outside core modules; breaking changes still ship between versions.

Security backlog (audit-confirmed, open)

- Metrics endpoints readable by any user; unauthenticated TTS GPU-release endpoint; unthrottled raw LLM proxy route - small set of auth gaps still open.
- Containers run as root with source bind-mounted read-write; no Redis password; no CSP/HSTS headers at the edge; Next.js 14 is EOL with unpatched advisories on branch.
- Public exposure requires operational discipline (close registration, restart after config changes, check who registered while the door was open).

Structural limits

- Answer quality = your model's quality: a local 27B model will trail Gemini-class hosted models on hard reasoning.
- Good UX wants a GPU (LLM + TTS); CPU-only setups are functional but slow - a real adoption barrier vs. zero-install SaaS.
- Self-hosting tax: you are the SRE (upgrades, backups, TLS, tunnels) - exactly the audience it targets, but a wall for everyone else.
- No connectors (web clip, Drive, Obsidian, arXiv) - ingest is upload-only today.
- Small ecosystem: effectively one maintainer, no community plugin layer; bus-factor risk.

6. Bottom Line

OpenNoteBox delivers, on your own hardware, roughly 80% of NotebookLM's core value plus two things NotebookLM lacks (reader-synced narration and FSRS-anchored review) and two things no competitor combines at all (self-hosted word-synced TTS + graph retrieval + citations-to-page in one product). Its trade is explicit: you accept self-hosting and SRE duties in exchange for total data custody and model choice. The engineering foundation is unusually solid for a pre-1.0 self-hosted project; the honest gaps are mobile/multilingual reach, team features, and finishing the security backlog before public exposure. As a personal or small-team research/study system with a spare GPU, it is one of the strongest options in the open-source field; as a drop-in NotebookLM replacement for a non-technical user, it is not there yet.

Sources

- Product facts & architecture: this repository (README.md, docker-compose.yml, backend/app, frontend/src, docs/), plus a six-track security audit conducted against the working tree (2026-09-03/06).
- ChatPDF features & limits: chatpdf.com (fetched 2026-09-06).
- NotebookLM: notebook.google (fetched 2026-09-06; JS-rendered page yielded limited detail).
- Self-hosted NotebookLM-clone landscape: GitHub repository search results retrieved via proxy (2026-09-06).
- AnythingLLM, Open WebUI, RAGFlow, Khoj, Anki/FSRS: general public knowledge as of the model's knowledge cutoff; star counts approximate - verify before external use.